Dave Pomerantz

Testing Error Handlers by Simulating Errors

Buggy error handlers often escape detection, or are tested only halfway. Here are some tools to help you finish the job.

As recently as two years ago, I had no systematic way to simulate errors, so I was never sure if my software could gracefully recover from a problem. If my program contained a buried code sequence like this:

```
try
{
   DBTable *pPersTable =
      new DBTable("Personnel");
   if (pPersTable == 0)
   {
      throw(E_OPENING_DB_TABLE);
   }
   DoUsefulWork(pPersTable);
}
catch(HRESULT hr)
{
   DBErrorRecovery(hr);
}
```

eleven of the above thirteen lines of code would be present only to detect and propagate a failure in `new`. How could I know if I made a mistake in those eleven lines before shipping the product to a customer?

Good programs can contain hundreds of these error handlers. Most of them respond to conditions that rarely occur and are difficult to force in normal use. If I run my program in an environment low in memory, I can force one or two errors. Repeated testing will usually trigger the same errors, especially when testing memory allocation failures. It's nearly impossible to force all the different errors with an external test.

In this article, I present a simple but powerful tool to simulate all the errors a program handles. You can use this tool to verify that your error handlers are working.

Dave Pomerantz is the owner of On Target Software, a consulting firm in Marshfield, MA, focusing on C++ development for Windows, especially Windows CE. The error simulator source code, sample program, Win32 DLL, and complete technical documentation are available at http://www.targetsoft.com.

After all, why bother writing an error handler if you can't be sure it works?

A Simple Example

Listing 1 shows a small program with four failure points: `new`, `malloc`, `AdjustValve`, and `UpdateDatabase`. Each of these functions might return an error, throw an exception, or both. After detecting a failure, the sample program responds by logging a message, skipping the remaining functions, and returning an error code. Listing 2 shows the same program with functions inserted to simulate errors. The functions are conditionally compiled to appear only in a debug version.

Simulating Allocation Failure

In the example below, taken from the sample program, the `SimErrMalloc` function simulates an allocation error immediately after a call to `malloc`:

```
pArray = (char *) malloc(100);
pArray =
   SimErrMalloc(SIMLINE, pArray);
if (pArray==0)
{
   printf("C allocation error\n");
   bOK = false;
}
else
{
   delete pArray;
}
```

`SimErrMalloc` is a conditionally compiled C++ template function. The release version compiles to a null function, eliminating all traces of the error simulator from production software. The debug version compiles the following template, which frees allocated memory and returns zero, to simulate a failed allocation.

```
#define SIMLINE  __FILE__, __LINE__

// Simulate failed C allocation
template <class T>
T *SimErrMalloc
      (const char *strFile,
       int nLine, T *pData,
       long nCount = 1,
       double nProb = 1.0)
{
   if ((pData != 0) &&
       SimErr(strFile, nLine,
          SIMERR_MEMORY_ERROR,
          nCount, nProb))
   {
      free(pData);
      pData = 0;
   }
   return pData;
}
```

`SIMLINE` is a simple macro that passes the first two parameters — the source file and line number — to `SimErrMalloc`. These parameters identify the location of the simulation function. The third parameter, `pData`, points to the allocated memory.

The last two parameters, `nCount` and `nProb`, apply only to a random test. They control the number of times to simulate an error at this location and the probability of an error each time. The default is to simulate the error once (`nCount = 1`), the first time through (`nProb = 1.0`). For now, I'll ignore these two parameters and save the discussion of random error simulation for later.

`SimErrMalloc` calls `SimErr` to determine whether or not to simulate an error. `SimErr` looks up the location, finds out how many times it has already simulated an error in that location, and returns `true` to simulate an error, or `false` to peacefully continue on.

If SimErr returns true, SimErrMalloc frees the allocated memory and returns zero, simulating a failed allocation.

Simulating Exceptions

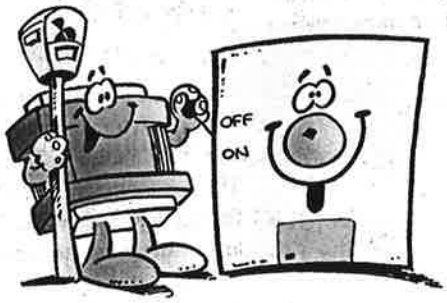
In addition to handling failure codes, most programs face the greater complexity of handling exceptions. In the sample, UpdateDatabase can return an error, but it can also throw an exception. The sample rigorously simulates both:

```
try
{
   ...
   if (bOK)
   {
      nResult = UpdateDatabase();
      SimErrException(SIMLINE,
         SIMERR_DATABASE_EXCEPTION);
      nResult =
         SimErrResult(SIMLINE,
            nResult,
            DATABASE_ERROR);
      if (nResult == DATABASE_ERROR)
      {
         printf("Update failed.\n");
         bOK = false;
      }
   }
}
....
```

continued on page 28

Software Protection

NEW! WIBU-BOX/U for USB



All the Extras You Need!

WIBU-KEY offers innovative features such as PayPerUse, Software Metering and License Management.
WIBU-BOX hardware for LPT, COM, USB, ADB, ISA and PCMCIA interfaces are functionally identical and interchangeable.
Profit from new ways of software distribution with WIBU-KEY!

Call Now for Your Free Test Kit
(800) 986 6578
www.wibu.com

ISO 9001 CERTIFIED

GRIFFIN TECHNOLOGIES
North and South America:
Griffin Technologies, LLC
1617 St. Andrews Drive, Lawrence, KS 66047
Tel.: (785) 832-2070 · Fax.: (785) 832-8787
Email: sales@griftech.com · www.griftech.com

International
WIBU-SYSTEMS AG
Rueppurrer Strasse 54
D-76137 Karlsruhe
Tel.: +49-721-93172-0
Fax: +49-721-93172-22
Email: info@wibu.com
http://www.wibu.com

WIBU SYSTEMS
Quality the World Trusts

Listing 1: *An example program with four failure points*

```
int main(int argc, char **argv)
{
   bool          bOK = true;  // keep going
                              // if true
   int           nResult;
   CSampleObject *pObject;
   char          *pArray;

   // Instantiate object with C++ allocation
   pObject = new CSampleObject;
   if (pObject==0)
   {
      printf("C++ allocation error.\n");
      bOK = false;
   }
   else
   {
      delete pObject;
   }
   // Allocate char array with malloc
   if (bOK)
   {
      pArray = (char *) malloc(100);
      if (pArray==0)
      {
         printf("C allocation error.\n");
         bOK = false;
      }
      else
      {
         delete pArray;
      }
   }

   try
   {
      if (bOK)
      { // Adjust real-time device
         //   note: AdjustValve can throw
         //         a CDeviceException
         bOK = AdjustValve();
         if (!bOK)
         {
            printf("The valve stuck.\n");
         }
      }
      if (bOK)
      { // Update remote-server database
         //   note: UpdateDatabase can throw
         //         a CDatabaseException
         nResult = UpdateDatabase();
         if (nResult == DATABASE_ERROR)
         {
            printf
               ("Database update failed.\n");
            bOK = false;
         }
      }
   }
   catch(CDeviceException *pDeviceException)
   {
      printf("Unrecoverable device error.\n");
      delete pDeviceException;
      bOK = false;
   }
   catch(CDatabaseException *pDBException)
   {
      printf
         ("Unrecoverable database error.\n");
      delete pDBException;
      bOK = false;
   }
   catch(...)
   {
      printf("Unknown exception.\n");
      bOK = false;
   }

   nResult = (bOK) ? 0 : -1;
   printf("Sample program exiting with return "
          "code = %d.\n", nResult);
   return nResult;
}
//End of File
```

continued from page 24

```
catch(CDatabaseException
   *pDBException)
{
   printf("Unrecoverable "
      "database error.\n");
   delete pDBException;
   bOK = false;
}
```

SimErrException throws an exception. SimErrResult sets nResult to DATABASE_ERROR.

Table 1 summarizes the error simulation functions and their parameters.

Systematic Testing

The simulation functions are simple. Inserting the function calls, though tedious, is also simple. The only tricky part is deciding when to

Listing 2: *The same program as Listing 1 with functions inserted to simulate errors*

```
int main(int argc, char **argv)
{
   bool          bOK = true;  // keep going if true
   int           nResult;
   CSampleObject *pObject;
   char          *pArray;

   SimErrCustomInit(SIMERR_TIME_SEED, SIMERR_SYSTEMATIC);

   // Instantiate object with C++ allocation
   pObject = new CSampleObject;
   pObject = SimErrNew(SIMLINE, pObject, SIMERR_FOREVER, .4);
   if (pObject==0)
   {
      printf("C++ allocation error.\n");
      bOK = false;
   }
   else
   {
      delete pObject;
   }

   // Allocate char array with malloc
   if (bOK)
   {
      pArray = (char *) malloc(100);
      pArray = SimErrMalloc(SIMLINE, pArray);
      if (pArray==0)
      {
         printf("C allocation error.\n");
         bOK = false;
      }
      else
      {
         delete pArray;
      }
   }

   try
   {
      if (bOK)
      { // Adjust real-time device
         //   note: AdjustValve can throw a CDeviceException
         bOK = AdjustValve();
         SimErrException(SIMLINE, SIMERR_DEVICE_EXCEPTION,
            4, 0.33);
         bOK = SimErrZero(SIMLINE, bOK, SIMERR_FOREVER, 0.2);
         if (!bOK)
         {
            printf("The valve stuck.\n");
         }
      }
      if (bOK)
      { // Update remote-server database
         //   note: UpdateDatabase can throw a CDatabaseException
         nResult = UpdateDatabase();
         SimErrException(SIMLINE, SIMERR_DATABASE_EXCEPTION);
         nResult = SimErrResult(SIMLINE, nResult, DATABASE_ERROR);
         if (nResult == DATABASE_ERROR)
         {
            printf("Database update failed.\n");
            bOK = false;
         }
      }
   }
   catch(CDeviceException *pDeviceException)
   {
      printf("Unrecoverable device error.\n");
      delete pDeviceException;
      bOK = false;
   }
   catch(CDatabaseException *pDBException)
   {
      printf("Unrecoverable database error.\n");
      delete pDBException;
      bOK = false;
   }
   catch(...)
   {
      printf("Unknown exception.\n");
      bOK = false;
   }

   nResult = (bOK) ? 0 : -1;
   printf("Sample program exiting with return code = %d.\n",
         nResult);

   SimErrExit(true);
   return nResult;
}
//End of File
```

C/BOOKS
Accounting System
Simple & Fast...
Delivering ...
√Customizable Accounting
√4GL Database
√Integrated Accounting
√Source Code
√Real-Time Processing
√Suite of nine modules
√Portable - Data/Applications
√C/C++ API
√ODBC Client/Server
√Visual Basic (API)
√Cost Effective
√Web Enabled
√Wysiwyg (Reporting)
√Unix (most versions)
√Windows NT/95
FREE DEMO Available!
Call - (800) 541-4580
CONETIC
SOFTWARE SYSTEMS, INC.

trigger the errors. If the first function always triggers an error, the program always exits before exercising other error handlers.

An error simulation function needs to turn itself off after its associated error handler has succeeded. To test the six error handlers in Listing 2, I should run the program six times. On each successive run, the previous error should be inactive and the next one should fire, until all errors have been simulated.

Each simulation function calls `SimErr` to determine whether or not it should trigger an error. `SimErr` maintains a log, by source file and line number, that tracks the number of the times the simulation has fired. Each time `SimErr` is called, it updates an internal copy of the log. At initialization, `SimErrInit` reads the simulation log and writes a backup copy. Before the program terminates, `SimErrExit` writes the simulation log.

After recovering from an error and exiting properly, the next run of the program will skip the previous failure and advance to the next error simulation. This permits successive program runs to proceed stepwise until each failure point has been tested.

If an error handler fails and the program aborts before calling `SimErrExit`, the updated log won't be written. This is important, because it allows you to repeat the same failure until you fix the error handler that's responding incorrectly.

It's possible for the program to exit successfully even if the error handler fails. For example, the error handler may have caught the error but displayed the wrong message. In this case, you need to manually copy the backup log file over the updated log file to repeat the same test.

Oh, the chore of testing!

It's certainly tedious testing several hundred error handlers, each of which causes the program to exit. It's worse if your program takes a long time to restart. Unfortunately, I don't think there's a simple answer. This kind of thorough testing is at the heart of quality assurance. Using an automated test framework to script the tests will make it easier, but error handlers account for approximately twenty percent [1] of your code. If your product is at all complex, testing and fixing twenty percent of it will take time.

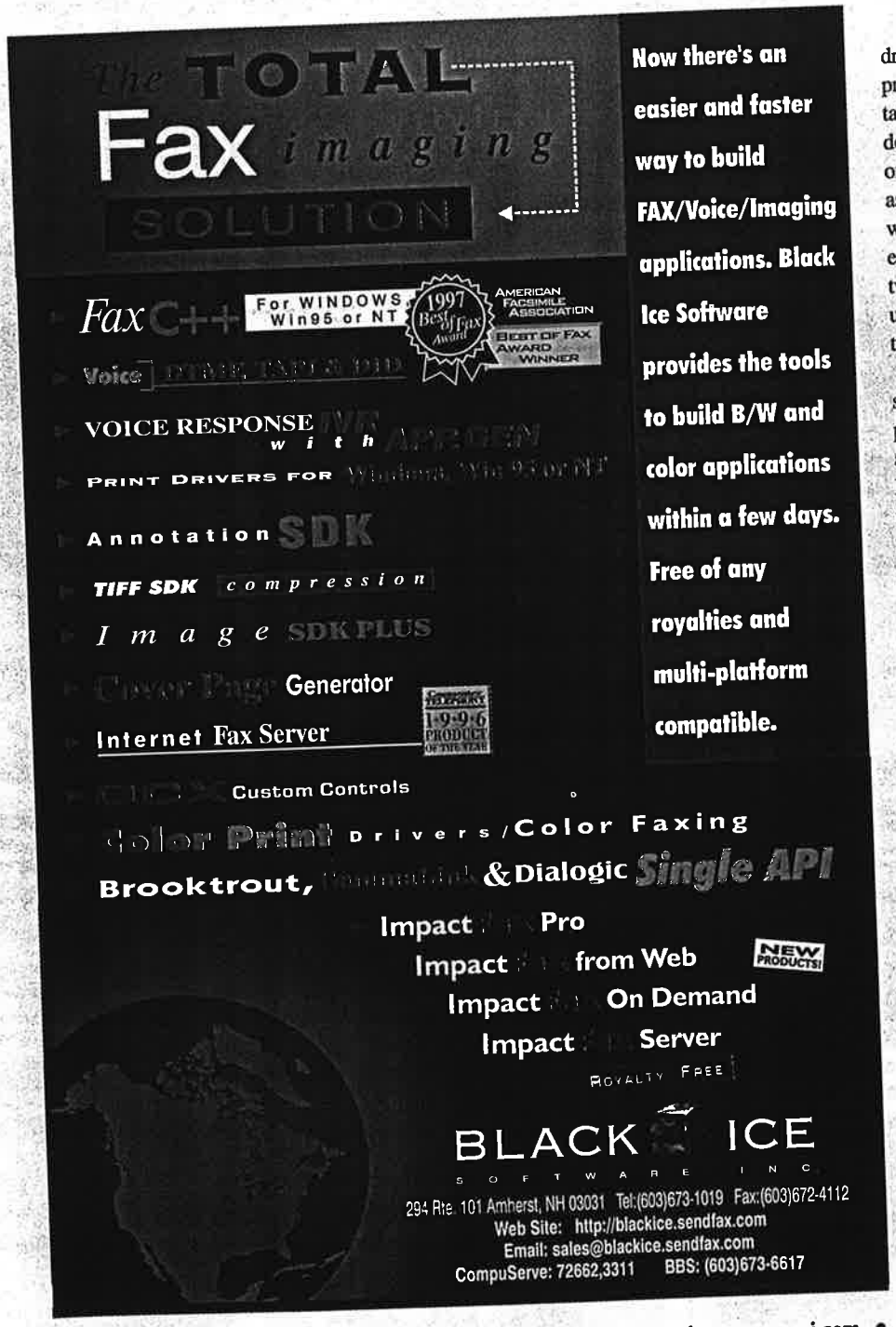
I prefer to do the testing piecemeal, at the same time that I unit-test each module. Testing periodically is a lot more tolerable than testing all at once, and I catch mistakes closer to when I made them. Besides, it's enormously satisfying to watch my software handle all kinds of system and hardware failures without blinking. Because of the propagation of errors, however, it's necessary at some point to test the complete integrated system, even if each module has been tested separately.

Random Testing

Once you've systematically tested all th error handlers, you'll have gone farthe than most programmers toward buildin bulletproof software, but the real world ha some nasty ordnance. The bullets don always come from where you're looking.

A systematic test triggers a failure at th first opportunity. A random test waits for th second, or perhaps the tenth or thousandt opportunity to trigger a failure. Because th state of the software may be different at th point, error handling that worked in a sy tematic test may fail in a random test.

The simulation template functions su as `SimErrMalloc` set the default count a probability to 1, which guarantees an er will be simulated once, the first ti through. By filling in these parameters, y can simulate the error more than once, e



time with a given likelihood. If you set the count to SIMERR_FOREVER, the error will be simulated indefinitely.

In the sample, the AdjustValve function is followed by a simulation function that will fail roughly one-third of the time it's executed until it has failed four times.

Table 1: *Error simulation functions*

Function	Purpose	Parameters
SimErrCustomInit	Customized initialization for a single target application. Calls SimErrInit.	type of test: random or systematic
SimErrInit	Create error simulation object and read records from prior testing.	structure containing log file name, random error seed, and callbacks
SimErrExit	Write records from this test and clean up.	whether to print a full report
SimErrEnable	turn on/off error simulation	whether to turn on or off
SimErrNew	simulate failure in C++ allocation	pointer to C++ object
SimErrMalloc	simulate failure in C allocation	pointer to allocated memory
SimErrResult	simulate failure of a called function that returns zero on success	returned value, error value
SimErrZero	simulate failure of a called function that returns false on failure	returned value
SimErrException	throw a system- or application-defined exception	exception type

```
bOK = AdjustValve();
SimErrException(SIMLINE, SIMERR_DEVICE_EXCEPTION, 4, 0.33);
```

If you change this function to:

```
SimErrException(SIMLINE, SIMERR_DEVICE_EXCEPTION,
   SIMERR_FOREVER, 0.0001);
```

the valve will return a failure indefinitely at the rate of one in ten thousand. This is useful for simulating intermittent failures during an extended stress test.

I typically test all the error handlers once for basic operation, then test again randomly. As a convenience, SimErrInit takes a parameter which turns random testing off. This forces all errors to be simulated exactly once, regardless of their parameters, making it easy to switch back and forth between systematic and random testing.

Error Tracing

Most debug environments have some form of trace output. The simulation functions direct all their output through a callback function, which you can tailor to your environment. Listing 3 shows the trace output from the first program run, which simulates a single memory error. The first line of the trace;

```
<SIMERR> Starting error simulator with a systematic test.
```

shows whether the test is random or systematic. If it were a random test, the trace would also show the seed used for the random numbers.

Training Videos for Developers
KeyStone Learning Systems Corp.
Microsoft Certified Solution Provider
Visual C++ 5.0
David Harding
Microsoft Certified & Visual C++ Instructor
KeyNotes Free
Average: 2 hours per video
●Visual C++ 5.0 Level 1 $89.95
Introduction to Developer Studio • Developer Studio • Wizards & Tools • Help
Project Settings • Multi Project Support • Customization • Single Document Interface
Overview • Creating a SDI Application with AppWizard • Using the Debugger
Introduction to Multiple document Interface • What is MDI?
●Visual C++ 5.0 Level 2 $89.95
Building the MDI Application • Adding a Second Document Template
The Document/View Architecture • Adding the User Interface
User Interaction with the View • Drawing Shapes • Multiple Views
●Visual C++ 5.0 Level 3 $89.95
Building the MDI Application cont'd • On Idle Processing • Optimization
Printing & Print Preview Techniques • Navigation & Command Routing
MFC Object Navitation • Command Routing • Creating Command Routers
MFC Foundation Classes • The Hierarch Chart • Components Gallery
●Visual C++ 5.0 Level 4 $89.95
Modal Dialogs • Overview • Modal Dialogs & DoModal() • Using Common Dialogs
Customizing Modal Diaologs • Overview • Creating Dialogs (Adding Controls)
Associating Data with Controls • Initialization • Controls
Introduction to ActiveX Controls • Overview of ActiveX Controls
●Visual C++ 5.0 Level 5 $89.95
Modeless Dialogs • Overview • Creating a Modeless Dialog • Adding Functionality
MFC Extension DLL's Overview Dialogs in DLL's • Creating an MFC DLL with AppWizard
Property Pages • Creating a Property Page & Resources • Internet Server API
●Visual C++ 5.0 Level 6 $89.95
Serialization • Principles of Serialization • Database Access • Snapshots & Dynasets
Data Access Objects • Creating Data Access Object Clases • The Debugger
Special-All Six $449.95
Special-Any Three $249.95
4 Easy Ways to Order:
CALL 1-800-581-8433
801-375-8680
FAX 1-801-373-6872
WEB www.klscorp.com/cuj
MAIL KeyStone Learning Systems Corp.
2241 Larsen Parkway • Provo, UT 84606
Domestic Shipping 2-3 Days: $5.00 for the first two videos, plus $1.50 for each additional video tape.
ADA CUJ98
30 DAY MONEY BACK GUARANTEE
We accept C.O.D., Company P.O. or check +

Before forcing an error, the simulator writes the source file and line number of the function along with the type of error:

```
SIMERR> at sample.cpp(42) *** SIMULATING MEMORY ERROR ***
C++ allocation error
Sample program exiting with return code = -1
```

In this case, the simulated failure caused the sample program to report the error and exit. Just before exiting, the program called `SimErrExit`, which wrote a detailed report divided into three sections:

- Errors that were simulated in this session and prior sessions. Errors in this session are marked NEW.
- Simulation functions that were encountered but not yet triggered. This can happen only during random testing.
- A summary count of simulated errors, organized by error type.

Listing 4 shows the output after six systematic runs. At this point, all errors have been simulated.

How the Simulator Works

The simulator tracks functions that call it. Each entry in this list contains the source file name, line number, error type, and number of times a function at that location has simulated an error. At initialization, the simulator reads the list from the log file and makes a backup copy of the file.

Each time a simulation function is executed, the simulator looks up the corresponding entry and decides whether or not to simulate a failure. To simulate an error, it returns `true`. To simulate an exception, the simulator invokes a callback routine that the target program provided to throw environment- and application-specific exceptions. At termination, the simulator writes the simulation log file and the summary report.

The list entries themselves are C++ objects and the list is an STL vector. The simulator is a singleton C++ object that, in the Windows implementation, resides in its own DLL.

Listing 3: *Error trace output from first program run*

```
<SIMERR> Starting error simulator with a systematic test.
<SIMERR> log file "errlog.txt" not found, test begins with first
<SIMERR> error
<SIMERR> atsample.cpp(42) *** SIMULATING MEMORY ERROR ***
C++ allocation error.
Sample program exiting with return code = -1.
<SIMERR>
<SIMERR> *** ERRORS SIMULATED ***
<SIMERR>
<SIMERR>     File                  Line Error-Type Requested Total New
<SIMERR> ---------------------------------------------------------------
<SIMERR> NEW sample.cpp             42     MEMORY   FOREVER     1   1
<SIMERR>
<SIMERR> *** ERROR SIMULATION TOTALS ***
<SIMERR>
<SIMERR> Type        Triggered Remaining Total
<SIMERR> ------------------------------------
<SIMERR> RESULT            0         0     0
<SIMERR> MEMORY            1         0     1
<SIMERR> DEVICE            0         0     0
<SIMERR> DATABASE          0         0     0
<SIMERR> ------------------------------------
<SIMERR> ALL TYPES         1         0     1
<SIMERR>
<SIMERR> Triggered 1 error(s) in this session.
```


Securely sell your software worldwide

Solutions from Rainbow, the world leader in secured software distribution

Sentinel Software Protection is the most advanced and reliable protection available. Over 25,000 developers rely on more than 13 million Sentinels to prevent piracy and ensure sales revenue.

SentinelLM Software gives customers new ways to try, buy, and use your products. It is the most flexible way to enhance and control your software licenses and distribution.

Sentinel Express is the fastest way to securely sell, distribute, activate, upgrade and register your software via the Internet.

Professional Services Group provides developers security consulting, custom programming and global distribution solutions.

Secure your success with a *FREE* brochure today!

Just call or visit our web site

(800) 705-5552

www.rainbow.com/success

RAINBOW TECHNOLOGIES

Securing Software Success

CALIFORNIA: (714) 450-7300 • FRANCE: (33) 1 41 43 29 00 • GERMANY: (49) 89 32 17 98 0 • U.K.: (44) 1932 579200 Distributors Located Worldwide • EMAIL: sentinel@rainbow.com

©1997 Rainbow Technologies, Inc. Sentinel, SentinelLM, Sentinel Express and Sentinel Professional Services Group are trademarks of Rainbow Technologies, Inc. All rights reserved.

Customizing the Simulator

The simulator source code is freeware. It's written in ANSI C++ and is available from `ftp://ftp.mfi.com/pub/cuj` and at `www.targetsoft.com`. The simulator was built with Visual C++ but could easily be adapted to any ANSI C++ compiler. Each program that uses the simulator needs a small source file to customize the error types and trace output. Alternatively, you could build the customization right into the simulator library, but I prefer the flexibility of allowing different error types in the target program.

The custom header and source files are shown in Listings 5 and 6. They provide:

- application-defined error and exception types
- a callback function to write the simulator messages to the debug log file
- a callback function to throw exceptions.

If you're not using C++, you have more work ahead of you. Fortunately, the simulator is only 1,200 lines of code and would therefore not be difficult to rewrite in another language. I feel the payoff would be worthwhile for all but the smallest projects. See the FAQ sidebar on what would be involved to re-implement in C.

Listing 4: *Error trace output after six systematic runs*

```
<SIMERR> Starting error simulator with a systematic test.
<SIMERR> backup log file written to "bkuplog.txt"
<SIMERR> atsample.cpp(87) *** SIMULATING RESULT ERROR ***
Database update failed.
Sample program exiting with return code = -1.
<SIMERR>
<SIMERR> *** ERRORS SIMULATED ***
<SIMERR>
<SIMERR>     File                  Line Error-Type Requested Total New
<SIMERR> ---------------------------------------------------------------
<SIMERR>     sample.cpp             42     MEMORY   FOREVER     1   0
<SIMERR>     sample.cpp             57     MEMORY         1     1   0
<SIMERR>     sample.cpp             75     DEVICE         4     1   0
<SIMERR>     sample.cpp             76     RESULT   FOREVER     1   0
<SIMERR>     sample.cpp             86   DATABASE         1     1   0
<SIMERR> NEW sample.cpp             87     RESULT         1     1   1
<SIMERR>
<SIMERR> *** ERROR SIMULATION TOTALS ***
<SIMERR>
<SIMERR> Type        Triggered Remaining Total
<SIMERR> ---------------------------------------
<SIMERR> RESULT              2         0     2
<SIMERR> MEMORY              2         0     2
<SIMERR> DEVICE              1         0     1
<SIMERR> DATABASE            1         0     1
<SIMERR> ---------------------------------------
<SIMERR> ALL TYPES           6         0     6
<SIMERR>
<SIMERR> Triggered 1 error(s) in this session.
```

Adding Environment-Specific Functions

Since the simulator DLL provides the error logging and triggering, the simulation functions need only five or six lines of code to simulate specific errors. This makes it easy to add functions for error conditions that are particular to an environment. For example, to simulate an error in a function that creates a Windows timer, you might use this code sequence:

```
m_nTimer = SetTimer(1000, // ID
                    100,  // interval
                    0);   // WM_TIMER
m_nTimer = SimErrZero(SIMLINE, m_nTimer);
```

When the error is simulated, `SimErrZero` will return zero. This causes a problem, however, since a Windows timer really did get created and is now dangling, unused and inaccessible.

For these kinds of system calls, it may be helpful to write a few functions like `SimErrTimer` to destroy the system resource that was created:

Listing 5: *Custom header file for application-specific error simulation*

```
enum ERROR_TYPE
{
   SIMERR_RESULT_ERROR,        // failing function returns
                               // an error code
   SIMERR_MEMORY_ERROR,        // failed allocation returns zero
   SIMERR_DEVICE_EXCEPTION,    // application-specific exception
   SIMERR_DATABASE_EXCEPTION,  // application-specific exception
   NUM_SIMERR_TYPES
};

#define SIMERR_MSGLEN   256

//———————— DEFINE IF SIMULATING ERRORS ————————
#ifdef SIMERR

bool SimErrCustomInit(int nSeed, SIMERR_TEST_TYPE testType);

//———————— DEFINE IF NOT SIMULATING ERRORS ————————
#else

inline bool SimErrCustomInit(int , SIMERR_TEST_TYPE )
{ return true; }

#endif // SIMERR
//End of File
```

Listing 6: *Custom source file for application-specific error simulation*

```
//------------------- DEFINE IF SIMULATING ERRORS -------------
#ifdef SIMERR

const char *SAMPLE_ERROR_LOG = "simerr.txt";

/*-----------------------------------------------------------
g_errorInfo    defines each error type and whether it throws
               an exception
-----------------------------------------------------------*/
static errorInfo_t g_errorInfo[] =
{
{ SIMERR_RESULT_ERROR,       "RESULT",   SIMERR_TYPE_ERROR      },
{ SIMERR_MEMORY_ERROR,       "MEMORY",   SIMERR_TYPE_ERROR      },
{ SIMERR_DEVICE_EXCEPTION,   "DEVICE",   SIMERR_TYPE_EXCEPTION  },
{ SIMERR_DATABASE_EXCEPTION, "DATABASE", SIMERR_TYPE_EXCEPTION  }
};

/*-----------------------------------------------------------
PrintSimulatorMessage  Redirect messages from error simulator
-----------------------------------------------------------*/
static void PrintSimulatorMessage(const char *pMsg)
{
   printf("<SIMERR> %s\n", pMsg);
}

/*-----------------------------------------------------------
ThrowSimulatorException  Throw an exception corresponding to
                         an error type.

The simulator calls this function to simulate an
application-specific exception.
-----------------------------------------------------------*/
static void ThrowSimulatorException(int nErrorType,
                                    const char *pSourceFilename,
                                    int nSourceLine)
{
   switch(nErrorType)
   {
   case SIMERR_DEVICE_EXCEPTION:
      {
      CDeviceException *pException = new CDeviceException();
      throw(pException);
      break;
      }
   case SIMERR_DATABASE_EXCEPTION:
      {
      CDatabaseException *pException = new CDatabaseException();
      throw(pException);
      break;
      }
   default:
      PrintSimulatorMessage("Invalid exception type");
   }
}

/*-----------------------------------------------------------
ValidateErrorTypes  Make sure error info array matches
                    ERROR_TYPE enum
-----------------------------------------------------------*/
static bool ValidateErrorTypes(void)
{
   bool bOK = true;
   int nErrorTypes = sizeof(g_errorInfo) / sizeof(errorInfo_t);
   if (nErrorTypes == NUM_SIMERR_TYPES)
   {
      for (int i=0; bOK && (i < NUM_SIMERR_TYPES); ++i)
      {
         if (g_errorInfo[i].m_nErrorType != i)
         {
            bOK = false;
         }
      }
   }
   else
   {
      bOK = false;
   }
   if (!bOK)
   {
      PrintSimulatorMessage
         ("error type enum does not match g_errorInfo.");
   }
   return bOK;
}

/*-----------------------------------------------------------
SimErrSampleInit  Allocate and initialize the sample
                  error simulator.
-----------------------------------------------------------*/
bool SimErrCustomInit(int nSeed, SIMERR_TEST_TYPE testType)
{
   bool bOK = false;

   // Make sure we didn't add an error type without
   // modifying g_errorInfo
   bOK = ValidateErrorTypes();

   if (bOK)
   {
      // Fill in information that initializes the simulator
      simerrInit_t init;
      init.m_nVersion     = SIMERR_VERSION;
      init.m_pFilename    = "errlog.txt";
      init.m_nSeed        = nSeed;
      init.m_testType     = testType;
      init.m_pfMsg        = PrintSimulatorMessage;
      init.m_pfThrow      = ThrowSimulatorException;
      init.m_pErrInfo     = g_errorInfo;
      init.m_nErrTypes    = NUM_SIMERR_TYPES;

      // Init simulator
      bOK = SimErrInit(&init);
   }
   if (!bOK)
   {
      PrintSimulatorMessage
         ("error simulator could not be initialized.");
   }
   return bOK;
}

#endif //SIMERR
//End of File
```

```
inline UINT
SimErrTimer(const char *strFile, int nLine, UINT nTimer,
   long nCount = 1, double nProb = 1.0)
{
   if ((nTimer != 0) &&
       SimErr(strFile, nLine, SIMERR_RESULT_ERROR,
          nCount, nProb))
   {
      KillTimer(0, nTimer);
      nTimer = 0;
   }
   return nTimer;
}
```

In my experience, a few of these functions — for timers, windows, files, and graphic objects — will handle your needs with a minimal amount of code.

Limitations and Caveats

Inaccurate Report

The simulator reports only the simulation functions that have called it. It's strictly a run-time analyzer, so it doesn't know about all the simulations that remain unexecuted in the target program. To get the count of simulation functions, I search all source files for all occurrences of `SIMLINE`. This is clumsy, but it works.

I've thought about writing a pre-processor to build the simulation log at compile time. That would, however, dramatically increase the size and complexity of the simulator and complicate its administration, so I've held off.

Editing Source Files

The simulation log persists between runs of the target program, tracking the line numbers of simulation functions. If you change a source file by inserting or deleting lines, an existing simulation function will have a different location in the file and will appear to the simulator to be a new function. Typically, the simulator will trigger it again, unnecessarily. Worse, it may never trigger a simulation function, assuming it had already been triggered.

To be safe, do one of the following:

1. Edit the log file to update the line numbers that have changed. This is the only viable option if you're in the middle of a long, complex test. The file is ASCII and is easy to edit.
2. Delete the simulator log file and restart the tests. If you've only just started testing, this is the best option.

This problem would be solved, of course, if the simulator had a preprocessor.

Conclusion

The error simulator is effective because it's simple and it solves a problem not addressed elsewhere. It takes time, however, to insert the simulation functions and more time to run the tests and fix the bugs. It's might seem hard to justify the time to test such unlikely errors. With projects such as embedded software and mission-critical applications, you have no choice. For other projects, you might consider the cost of finding and fixing one intermittent data-corruption bug that occurs when an out-of-memory condition triggers an obscure error handler. In that case, it may be cheaper to fix all the

This is NOT your Grandfather's Dongle

In anciet times, (PC history, circa: 1981-1982) the engineers at Data Security pioneered the first PC software protection dongle. Back then, they had to choose between the serial port and the parallel port as a place to attach a protection device. They chose the parallel port, because dot matrix printers were the only other peripherals using it.

Today, (circa: 1998), parallel ports are crowded, so a more private place for dongles is needed to limit the frustrating problems users experience in the field when connecting scanners, zip drives, network adapters, bi-directional printers and a myriad of protection dongles. As a result of developer demand, we are proud to introduce SecuriKey® KB, a dongle that attaches NOT to the parallel port but rather to the KEYBOARD port.

- Attaches to Key Board Port
- Transparent to User
- Works on Laptops
- Easy to Implement

To get your hands on a Developer Kit... Call Today

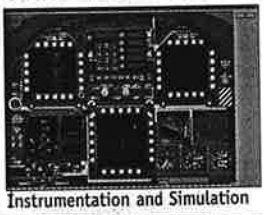
DATA SECURITY CORPORATION
150 Wright Brothers Dr, #560
Salt Lake City, UT 84116
[800] 793-5150
[801] 575-6600
www.datasec.com

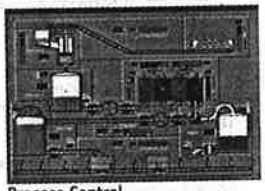
Accelerate Your Dynamic Graphics Development

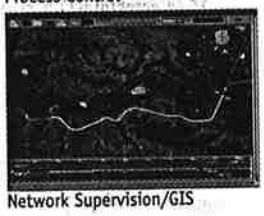
LOOX, LOOX++, LOOXWin

- Rapid creation of high-performance dynamic graphics screens
- Reuse of graphics objects and/or entire screens
- Cross-platform (Windows and Unix), C++, C and Ada APIs
- Integration with standard tools (Motif GUI builders, MFC, OCXs)

CALL US TODAY and ask about our free evaluation program.


Instrumentation and Simulation


Process Control


Network Supervision/GIS

Accelerating Graphics Development™

LOOX Software, Inc.
4962 El Camino Real, Ste. 206,Los Altos, CA 94022
T:650-903-0942 F:650-903-9824 • www.loox.com

error handlers in controlled tests during development than to fix a single intermittent bug at a customer site.

Note

[1] I admit, that's a wild guess. I haven't seen hard numbers on this, so I counted lines in my own software. Excluding comments, I found that testing, propagating, and responding to errors made up between 7 and 34 percent of the code, averaging about 20 percent. The percentage is lowest in modules that have few external interfaces, and is highest in I/O modules and other routines that interact heavily with the system. ❑

Frequently Asked Questions

What is the performance cost?

When you're debugging the error handlers, the performance cost is probably irrelevant, since the program will soon enter an error handler. In the release version, there is no performance cost. I've inspected the disassembled output of the Visual C++ optimizing compiler, and it generates no code for the null templates.

Isn't it enough to test failures at the low-level functions?

Error handlers are all over a program, but only a few low-level functions originate most errors.

Isn't it enough to simulate errors in these functions?

No. In addition to testing error detection, you need to test error propagation and recovery. That means testing all paths for all errors, not just all errors. The safest way is to simulate an error in every place that you test for an error.

You used templates and classes. How can I implement this in C?

I used templates to make the simulation functions typesafe and to allow default parameters. Instead, I could have used macros such as:

```
// Simulate failed C allocation
#define SimErrMalloc(pVoid)          \
{                                    \
   if ((pVoid) &&                    \
       SimError(__FILE__,            \
                __LINE__,            \
                MEMORY_ERROR,        \
                1, 1.0))             \
   {                                 \
      free(pVoid);                   \
      pVoid = 0;                     \
   }                                 \
}
#define SimFailMallocRandom(pVoid,\
                            nCount,\
                            nProb) \
{                                  \
   if ((pVoid) &&                  \
       SimError(__FILE__,          \
                __LINE__,          \
                MEMORY_ERROR,      \
                nCount,            \
                nProb))            \
   {                               \
      free(pVoid);                 \
      pVoid = 0;                   \
   }                               \
}
```

Instead of the one template, I had to define two macros: one for systematic errors and one for random errors. Macros have the advantage of letting you hide the __FILE__ and __LINE__ parameters directly in the macro.

I used object-oriented programming because I find it makes the code easier to develop and debug. In C, the same result could be accomplished with a module, static data, and static functions.

Aren't there any commercial tools that simulate errors?

The only tool I could find that claimed to support error simulation was Panorama by International Software Automation (www.softwareautomation.com). They discuss automatic error simulation on their website, but when I contacted the company, they recommended manual error simulation much like the material I've presented here. I also contacted NuMega Technologies, developers of BoundsChecker, but they do not have an error simulation tool for C++.

I think this is an excellent area for companies like ISA and NuMega to pursue. More sophisticated tools that use preprocessors can invisibly insert the simulation code during compilation and solve many of the problems associated with my very simple approach. ❑

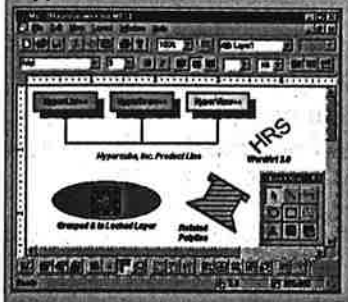
MFC Extension Class Libraries

HyperView++: VBA-like embeddable forms environment

Embeds a custom forms builder / run time system in your app. HyperView++ is a great, MFC native, VBA alternative. Fully customizable editor supports all Win and ActiveX controls, lightweight GUI objects, VBScripting, C++ actions and works with ODBC. Editor is more powerful than VBA's (adds in rotation, layers, line snapping) and is a superset of HyperDraw++. Can be compiled as an ActiveX control for use with VB, IE4. Over 150 licensees worldwide. Includes full source code, no royalties.

"We evaluated using VBA to implement our applets, but chose HyperView++ since it is lighter, faster and far more customizable."
Craig Codling, VP Development, InfoDynamics

HyperDraw++: VISIO-like drawing and diagramming classes



Adds professional drawing & diagramming functionality in your app. Features: pinup property sheet, groups, layers, zoom, line snapping for entity relationships, rotation, rulers & guides, format toobars, undo/redo, thumbnails, auto-save, OLE objects, printing and more. Fully customizable: add new objects and properties, restrict or expand editor. Upgrade to HyperView++ to make drawings come alive via data source connectivity, VBScripting, C++ actions, shortcut menus, hotspots, tooltips, context help.

HYPERCUBE
(800) 268-1812, harlan@hcube.com
http://www.hcube.com
FREE DEMOS!
10542 Bradbury Road, Los Angeles, California 90064 • (310) 559-2354